

Guía rápida para laboratorio de APIs REST con Spring Boot

Hecho para que puedas copiar y pegar comandos/código durante el laboratorio. Está basado en tus archivos de Clase 9.1, 9.2, documentación de productos y Clase 10.1.

1. Idea principal: ¿qué estás haciendo?

Tu proyecto Spring Boot levanta un servidor local en localhost:8080. Ese servidor tiene rutas o endpoints que responden datos en JSON. Por ejemplo, cuando entraste a `http://localhost:8080/product`, el navegador hizo una petición GET y la API respondió una lista de productos.

API no es Postman. API es tu servidor/ruta. Postman, consola del navegador, curl o Thunder Client son solo clientes para probarla.

```
Cliente / navegador / consola / curl
    |
    v
GET/POST/PUT/DELETE http://localhost:8080/product
    |
    v
Spring Boot Controller -> Repository -> Base de datos Northwind
    |
    v
Respuesta JSON
```

2. Palabras que debes saber sí o sí

Concepto	Significa
Endpoint	Ruta de la API. Ejemplo: <code>/product</code>
GET	Consultar datos. Ejemplo: listar productos
POST	Crear un dato nuevo. Ejemplo: crear producto
PUT	Actualizar un dato existente
DELETE	Eliminar un dato
JSON	Formato llave-valor que viaja entre cliente y servidor
Controller	Clase Java que recibe las peticiones HTTP
Repository	Clase/interfaz que consulta o guarda en la BD
Entity	Clase Java que representa una tabla de la BD

3. Endpoints importantes del proyecto

GET	<code>http://localhost:8080/product</code>	-> lista productos
GET	<code>http://localhost:8080/product/1</code>	-> obtiene producto con id 1
POST	<code>http://localhost:8080/product</code>	-> crea producto con JSON
POST	<code>http://localhost:8080/product?fetchId=true</code>	-> crea y devuelve el id
PUT	<code>http://localhost:8080/product</code>	-> actualiza producto
DELETE	<code>http://localhost:8080/product?id=1</code>	-> elimina producto con id 1
GET	<code>http://localhost:8080/category</code>	-> lista categorías
GET	<code>http://localhost:8080/supplier</code>	-> lista proveedores

Ojo: en este código específico, el PUT está configurado para recibir `x-www-form-urlencoded`, no JSON. Por eso para actualizar con consola usamos `URLSearchParams`.

4. Verificar que la API está viva

Abre el navegador y entra a:

http://localhost:8080/product

Si ves muchos datos con corchetes [] y llaves { }, significa que tu API respondió JSON correctamente.

5. Crear producto desde Console del navegador

Como no usarás Postman, hazlo desde la consola del navegador. Abre la página de productos, presiona F12, entra a Console y pega esto:

```
fetch("http://localhost:8080/product?fetchId=true", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    productName: "Producto prueba consola",
    supplier: {
      id: 1
    },
    category: {
      id: 1
    },
    quantityPerUnit: "10 unidades",
    unitPrice: 25,
    unitsInStock: 20,
    unitsOnOrder: 0,
    reorderLevel: 5,
    discontinued: false
  })
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

Respuesta esperada:

```
{
  "estado": "creado",
  "id": 78
}
```

El ID puede salir diferente. Lo importante es que diga estado: creado.

6. Comprobar que se creó

Vuelve a consultar la lista:

http://localhost:8080/product

Busca el nombre:

Producto prueba consola

También puedes consultar por ID. Cambia 78 por el ID que te devolvió tu API:

http://localhost:8080/product/78

7. Listar productos desde Console

Esto hace lo mismo que abrir la URL en el navegador, pero desde JavaScript:

```
fetch("http://localhost:8080/product")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

8. Obtener un producto por ID desde Console

```
fetch("http://localhost:8080/product/1")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

Si el ID existe, debe responder algo parecido a:

```
{
  "result": "ok",
  "producto": {
    "id": 1,
    "productName": "Chai"
  }
}
```

9. Actualizar producto desde Console

En este proyecto, el PUT recibe formulario, no JSON. Usa este código y cambia el ID por el producto que creaste:

```
fetch("http://localhost:8080/product", {
  method: "PUT",
  headers: {
    "Content-Type": "application/x-www-form-urlencoded"
  },
  body: new URLSearchParams({
    id: "78",
    productName: "Producto actualizado consola",
    unitPrice: "30",
    unitsInStock: "50"
  })
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

Respuesta esperada:

```
{
  "result": "ok"
}
```

Importante: cambia 78 por el ID real que te salió al crear el producto.

10. Eliminar producto desde Console

Cambia 78 por el ID real:

```
fetch("http://localhost:8080/product?id=78", {
  method: "DELETE"
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.error(error));
```

Respuesta esperada:

```
{
  "result": "ok"
}
```

11. Consultar categorías y proveedores

Antes de crear productos, conviene saber qué supplier y category existen:

```
fetch("http://localhost:8080/category")
  .then(response => response.json())
  .then(data => console.log(data));

fetch("http://localhost:8080/supplier")
  .then(response => response.json())
  .then(data => console.log(data));
```

Si usas supplier id 1 y category id 1, normalmente funcionan con Northwind.

12. Lo que debes explicar si te pregunta el profe

GET /product

- Sirve para listar todos los productos.
- No recibe body.
- Devuelve un arreglo JSON.

POST /product

- Sirve para crear un producto.
- Recibe un JSON en el body.
- Usa @RequestBody para convertir el JSON en un objeto Product.
- Guarda con productRepository.save(product).
- Devuelve HTTP 201 Created.

GET /product/{id}

- Recibe el id desde la URL con @PathVariable.
- Busca el producto con productRepository.findById(id).
- Si existe, devuelve result ok y el producto.
- Si el id no es número, devuelve error 400.

PUT /product

- Sirve para actualizar.
- En este proyecto consume x-www-form-urlencoded.
- Solo actualiza campos enviados.

DELETE /product?id=ID

- Recibe el id como query param con @RequestParam.
- Si existe, elimina con deleteById(id).

13. Errores comunes y solución rápida

Problema	Qué revisar
No abre localhost:8080	El proyecto Spring Boot no está corriendo o falló al levantar.
Error de conexión a BD	Revisa application.properties: BD northwind, usuario root, clave root.
POST devuelve error	Revisa que el body sea JSON válido y tenga Content-Type application/json.
PUT devuelve 415 o no funciona	Recuerda: este PUT usa x-www-form-urlencoded, no JSON.
No encuentro el producto creado	Usa fetchId=true y luego busca por ese ID.
supplier/category falla	Verifica que existan con GET /supplier y GET /category.

14. Chuleta final para memorizar

GET = leer datos
POST = crear datos
PUT = actualizar datos
DELETE = borrar datos

JSON = formato de datos
API = servidor/rutas que responden datos
Cliente = herramienta que llama a la API: navegador, consola, curl, Postman, etc.

Tu ruta principal:
<http://localhost:8080/product>

15. Frase lista para decir en clase

Al entrar a localhost:8080/product estoy consumiendo un endpoint REST con método GET. El servidor Spring Boot consulta la base de datos Northwind mediante el repository y devuelve una lista de productos en formato JSON. Para crear un producto uso POST, enviando un body JSON desde la consola del navegador.